

GrabThePhisher

CyberDefenders Blue Team Lab -- Threat Intelligence

Category: Threat Intel | Difficulty: Easy | Tool: Text Editor

MITRE ATT&CK Tactics: Initial Access, Exfiltration

The Scenario (in simple words)

A decentralized finance (DeFi) platform got reports of unauthorized cryptocurrency withdrawals from user accounts. Investigators found a fake website impersonating the real PancakeSwap exchange, designed to trick victims into typing in their wallet's secret recovery phrase. The fake site's source code (a "phishing kit") was recovered from a compromised server. Our job was to read through that code and figure out exactly how the scam worked and where the stolen data was being sent.

Background: what is a seed phrase, and why do phishing kits target it?

A cryptocurrency wallet's **seed phrase** (usually 12 or 24 ordinary words) is the master key to that wallet -- anyone who has it can fully restore and control the wallet's funds from anywhere, with no other password needed. Phishing kits impersonate popular wallet tools like **MetaMask** and present a fake "restore your wallet" or "verify your account" form, tricking victims into typing their seed phrase directly into an attacker-controlled page.

What Actually Happened (the story)

The phishing kit was built to look like it was restoring a **MetaMask** wallet, asking victims to enter their secret recovery phrase to "unlock" their account. The core logic lived in a file called **metamask.php**, written in **PHP**, a common server-side scripting language for web forms. When a victim submitted their phrase, the script also looked up the victim's approximate location using a geolocation service called **Sypex Geo**, adding country and city information to make the stolen data more useful to the attacker.

The script then exfiltrated the stolen phrase two different ways at once: it sent the data instantly to a private **Telegram** bot (using a hardcoded bot token and chat ID baked directly into the code), and it also appended a copy to a local log file (`log.txt`) on the compromised server as a backup. That log file was recovered and showed **three** separate victims had already had their seed phrases stolen by the time of the investigation. The kit's source code also contained a signed message from its developer, crediting a handle used to identify them within their criminal community.

Step-by-Step Investigation

Step 1 -- Explore the kit's file structure. Unzipped the provided archive and reviewed the folder layout -- noticed folders named after the impersonated brand and the server-side script location.

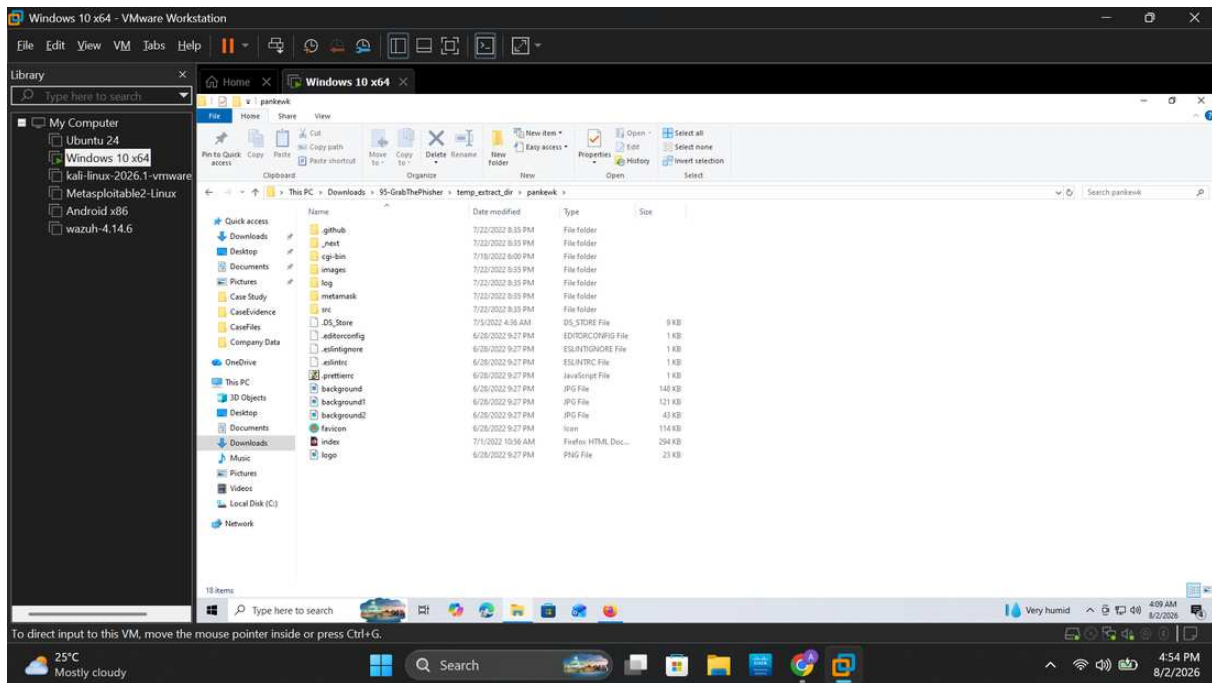


Fig 1 -- The phishing kit's folder structure, including a metamask folder (the impersonated wallet) and a log folder (harvested data).

Step 2 -- Identify the core phishing script. Opened the metamask folder and found a single .php file alongside the fake HTML page -- confirming both the wallet being impersonated and the file containing the actual malicious logic.

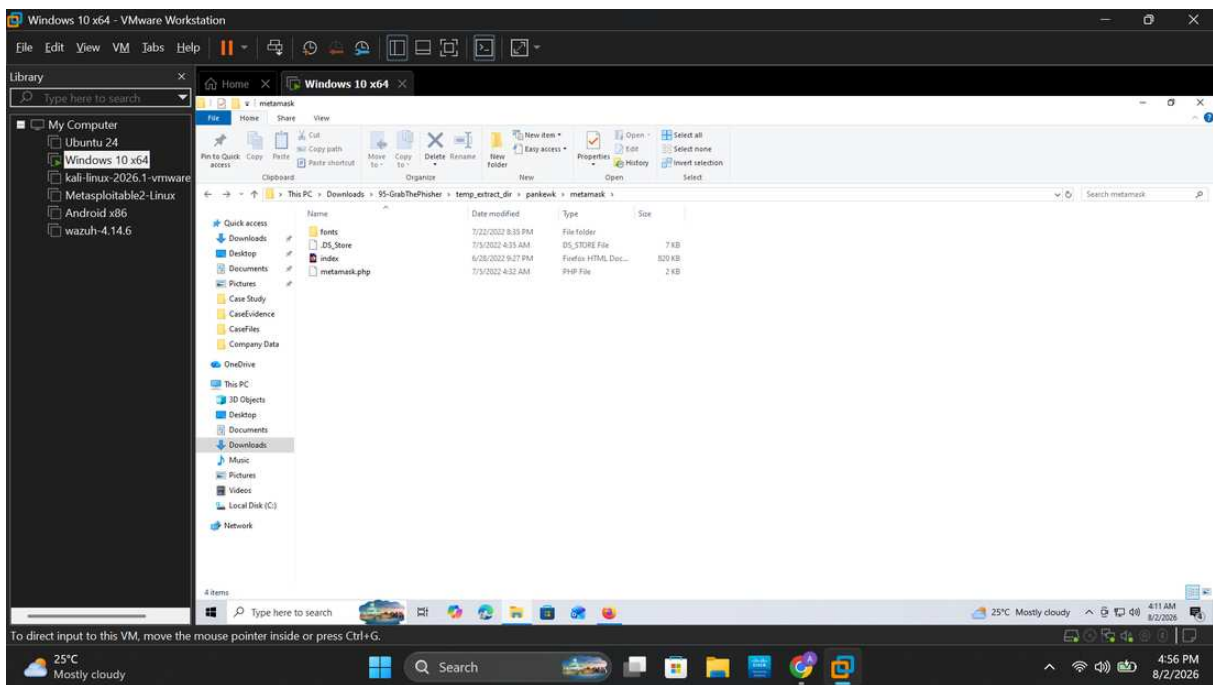


Fig 2 -- Inside the metamask folder: metamask.php is the only server-side script, responsible for handling stolen data.

Step 3 -- Read the phishing script's source code. Opened metamask.php in a text editor. The code revealed the geolocation lookup, the developer's signed comment, the message template built from the stolen phrase, and the exfiltration function sending data to both Telegram and a local log file.

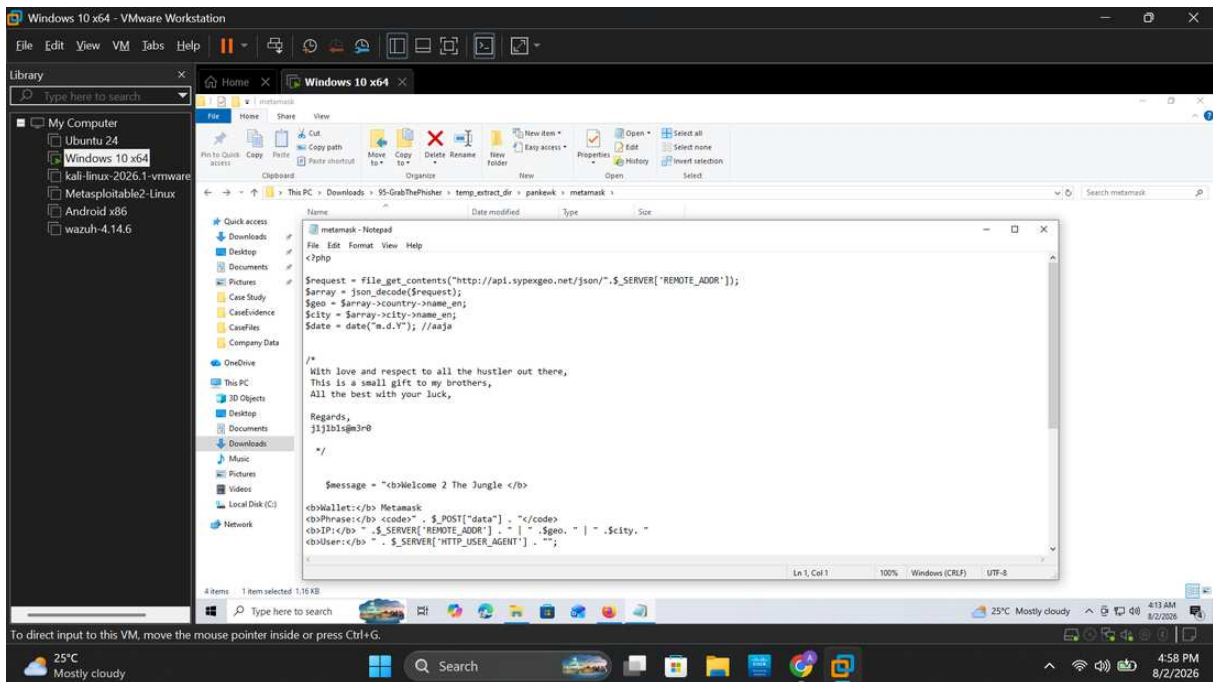


Fig 3 -- The top of metamask.php: the Syplex Geo lookup, the developer's signed comment block, and the start of the stolen-data message template.

Step 4 -- Locate the harvested data. Navigated to the log folder referenced in the PHP code and opened log.txt directly to see the actual stolen seed phrases.

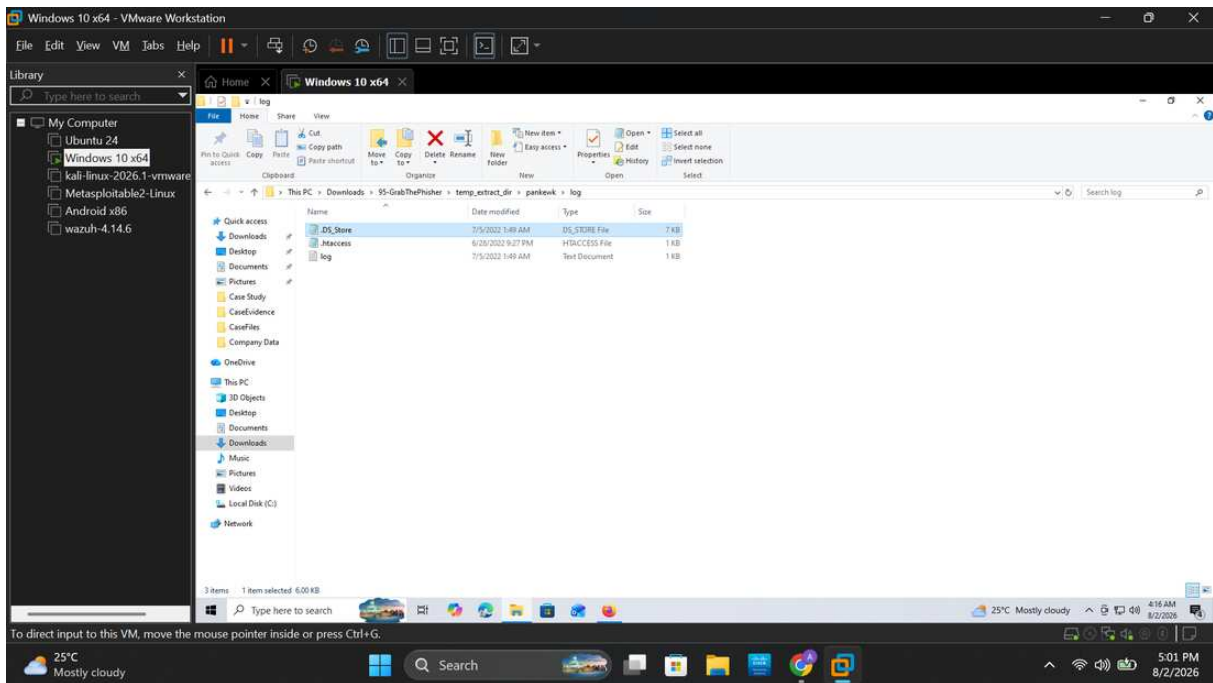


Fig 4 -- The log folder, containing the log.txt file where every stolen seed phrase gets appended.

Step 5 -- Read the collected seed phrases. Opened log.txt and found three separate lines, each a complete 12-word seed phrase from a different victim, with the most recent theft appearing on the last line.

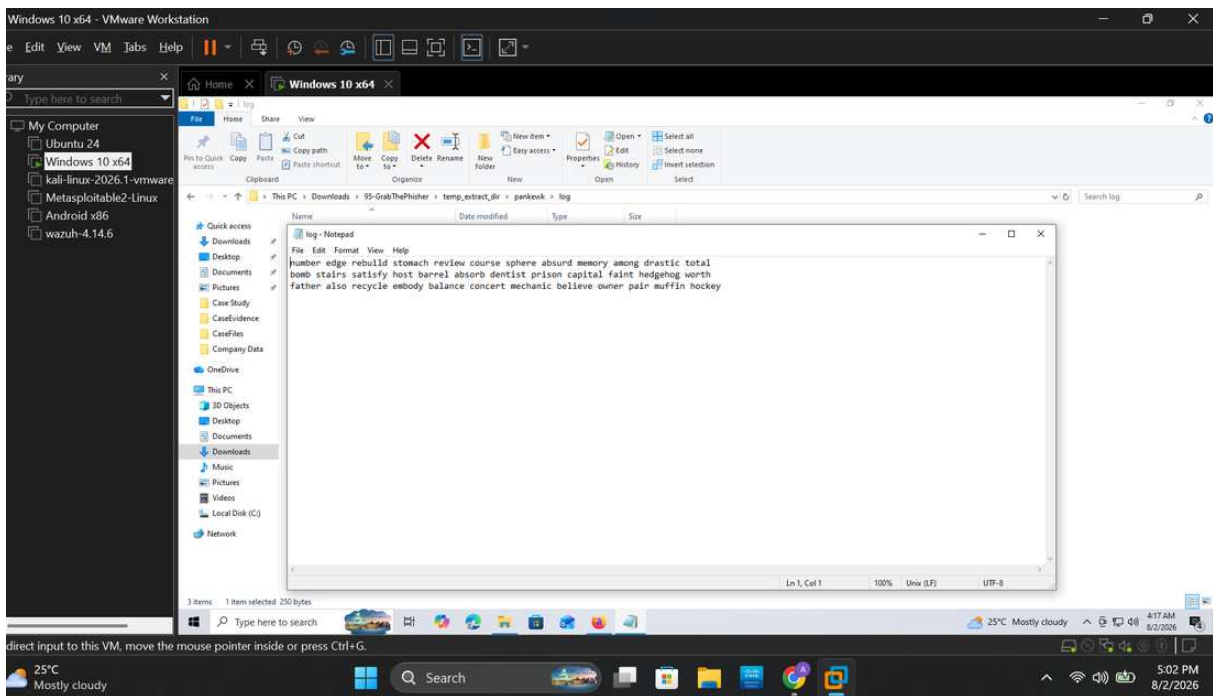


Fig 5 -- log.txt contains three stolen seed phrases; the bottom line is the most recently captured one.

Final Answers

Q	Question (short)	Answer
Q1	Wallet used for the seed phrase prompt	MetaMask
Q2	File containing the phishing kit's code	metamask.php
Q3	Language the kit was written in	PHP
Q4	Service used to retrieve victim machine info	Sypex Geo
Q5	Number of seed phrases already collected	3
Q6	Most recent seed phrase	father also recycle embody balance concert mechanic believe o
Q7	Medium used for credential dumping	Telegram
Q8	Token for accessing the channel	5457463144:AAG8t4k7e2ew3tTi0IBShcWbSia0Irvxm10
Q9	Chat ID for the phisher's channel	5442785564
Q10	Alias of the phish kit developer	j1j1b1s@m3r0

How Each Answer Was Found

Q1 -- MetaMask	The kit's folder was literally named metamask, and the fake HTML page inside it asked users for their MetaMask wallet's secret recovery phrase.
Q2 -- metamask.php	Only one .php file existed inside the metamask folder -- the .php extension marks it as the server-side script handling form submissions.
Q3 -- PHP	Directly inferred from the .php file extension, which is unique to the PHP scripting language.
Q4 -- Sypex Geo	Read directly from the source code: a request to the domain api.sypexgeo.net, used to look up the victim's IP-based location.
Q5 -- 3	Opened log.txt directly and counted the number of separate lines/entries -- each line is one stolen seed phrase.
Q6 -- seed phrase	The last (bottom) line in log.txt, since new entries are appended to the end of the file over time.
Q7 -- Telegram	The sendTel() function in the code calls the Telegram Bot API (api.telegram.org) to deliver stolen data -- confirmed by the domain name and the token/chat ID variables used.
Q8 -- Bot token	Hardcoded directly in the source code as the \$token variable inside the sendTel() function.
Q9 -- Chat ID	Hardcoded directly in the source code as the \$id variable inside the sendTel() function.
Q10 -- Developer alias	Found in a comment block left in the source code by the developer, signed off with "Regards, j1j1b1s@m3r0".

Personal learning log from the CyberDefenders SOC Analyst Tier 1 track. All lab data belongs to CyberDefenders; this document reflects my own investigation process and findings.